RouteLogger リスク分析

バージョン: RLog-v10.7 **作成日:** 2026年5月1日 **対象:** Firebase Hosting 構成（Vercel デプロイは別プロジェクトで実施予定のため本書では対象外）

---

0. サマリ

0.1 想定運用

- ホスティング: Firebase Hosting
- バックエンド: Firebase（Firestore + Storage + Anonymous Auth + Cloud Functions）
- ソース管理: GitHub
  - `origin`: `sato4app/RouteLogger-dev`（**Public**）
  - `production`: `sato4app/RouteLogger`（**Public**）

0.2 最重要結論

- **Firebase API キーが Public リポジトリに公開＋Firestore/Storage Rules が `request.auth != null` のみ＋Cloud Functions の認可が `context.auth` の有無のみ** という3点が連鎖し、攻撃者が API キーを取得 → 匿名サインイン → 全ユーザーのデータ閲覧・改ざん・削除・任意の Cloud Function 起動まで実行可能な状態にある。**P0 三点を同時に塞ぐ必要がある。**

0.3 優先度サマリ

優先度	項目
🔴 P0	`firebase-config.js` の Git 履歴除去＋API キー再発行
🔴 P0	Firestore / Storage Rules の厳格化
🔴 P0	Cloud Functions の認可強化（管理者判定）
🟠 P1	外部 KML/GeoJSON のサニタイズ（XSS）
🟠 P1	写真ライトボックス URL の許可リスト化
🟡 P2	Service Worker のキャッシュ＆更新ハンドリング
🟡 P2	データ消失動線の確認 UI 強化
🟡 P2	CDN ライブラリへの SRI 付与
🟡 P2	位置情報プライバシー＋退会フロー
🟢 P3	`viewport` のズーム禁止 / `functions/.env` 管理 / main 直プッシュ運用

---

1. 🔴 P0: 認証情報・認可の根本リスク（即時対応）

1.1 `js/firebase-config.js` がコミット済み＋Public リポジトリ

事実関係

- `.gitignore` のコメントには「APIキーを含むため」とあるが、当該行は `# firebase-config.js` と**コメントアウトされており除外されていない**
- `git ls-files` の結果、`js/firebase-config.js` は **追跡対象**
- `RouteLogger-dev` / `RouteLogger` の両 GitHub リポジトリとも **Public**（HTTP 200 を返す）
- Firebase Web API キー、`projectId=walklog-sato`、`appId` 等が世界中に露出

影響

- API キーが取得できる
- Firebase API キー単体は本来「公開前提」の仕様だが、本アプリは §1.2 のルールが緩いため、**事実上の認証バイパス手段**になっている
- 仮にいま `.gitignore` を修正しても、**Git 履歴・GitHub のフォーク・既存クローン・Web archive には残り続ける**

対応

1. **Firebase Console で当該 Web アプリの API キーを再発行**（旧キーは失効）
2. `js/firebase-config.js` を更新し、再デプロイ
3. `.gitignore` の `# firebase-config.js` のコメントアウトを外す
4. `git filter-repo` または BFG Repo-Cleaner で全履歴から `js/firebase-config.js` を削除
    - 両リポジトリ（dev / production）と全クローン保有者の協力が必要
5. リポジトリの Private 化を検討（履歴漏えいは取り返せないが、追加流出は止まる）

1.2 Firestore / Storage Rules の過剰許可

現状（`docs/firestore.rules`）

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {
    match /projects/{projectId} {
      allow read, write: if request.auth != null;
      match /{document=**} {
        allow read, write: if request.auth != null;
      }
    }
    match /tracks/{trackId} {
      allow read, write: if request.auth != null;
      match /{document=**} {
        allow read, write: if request.auth != null;
      }
    }
  }
}
```

現状（`docs/storage.rules`）

- `tracks/{allPaths=**}` も同様に `request.auth != null` で全許可

問題

- 匿名認証だけで `tracks` 全件を read/write 可能
- 攻撃者が API キーで匿名サインイン → 全ユーザーの GPS 軌跡・写真・メールアドレスを取得・改ざん・削除できる
- `userAdmin` コレクションのルールは `firestore.rules` に未記述
  - デフォルトの deny ならアプリが動かないため、本番では暗黙的に同等の緩いルールが適用されている可能性が高い
  - もし緩く運用されていれば、他人の `username` ドキュメントを上書きして **なりすまし**が可能

影響例

- 他ユーザーの自宅・行動範囲が露出 → 個人特定リスク
- 写真の盗難（メモ・撮影位置含む）
- メールアドレスの漏えい（PII）
- `tracks` の全件削除や書き換え
- なりすましによる第三者プロジェクト保存

対応（推奨ルール例）

```
match /tracks/{trackId} {
  allow read:   if request.auth != null && resource.data.userId ==
request.auth.uid;
  allow create: if request.auth != null && request.resource.data.userId ==
request.auth.uid;
  allow update, delete: if request.auth != null && resource.data.userId ==
request.auth.uid;
}
match /userAdmin/{username} {
  allow read:   if request.auth != null && resource.data.uid == request.auth.uid;
  allow create: if request.auth != null && request.resource.data.uid ==
request.auth.uid;
  allow update: if request.auth != null && resource.data.uid == request.auth.uid;
  allow delete: if false;
}
```

- 共有ユースケースが必要な場合は、`shared: true` フィールド等で限定的に開放する設計を検討
- `firebase deploy --only firestore:rules,storage` で反映

1.3 Cloud Functions の認可が「匿名でも通る」

現状

- `functions/index.js` の `generateKmzAndSendEmail` / `migrateRoutesToDrive` は `context.auth` の有無のみチェック
- **匿名認証でも通過**するため、API キーさえあれば誰でも呼べる

影響

- `generateKmzAndSendEmail({ projectName: '<任意>' })`
  - 任意ユーザーの登録メールアドレス宛にスパム的な KMZ 送信トリガー（送信先は `userAdmin` から引くため攻撃者には見えないが、メール送信スパムとして悪用可）
- `migrateRoutesToDrive({ all: true })`
  - **誰でも全ルートの Drive 移行を起動可能**（タイムアウト 540 秒・1GB メモリ）
  - 課金スパイク・Drive ストレージ消費・Drive API クォータ枯渇を簡単に引き起こせる
- 設定 UI 上は「画面設定等（隠し）」で隠蔽されているが、隠し UI ≠ サーバ認可ではない

対応

- `userAdmin/{username}.role == 'admin'` を Firestore で確認、または Firebase カスタムクレームで管理者権限を判別
- レート制限（同一 UID あたり N 回/日）を Cloud Functions 側に追加
- `migrateRoutesToDrive` は管理者専用とし、`projectName` 必須化＋`all`/`prefix` 廃止も検討

---

2. ◍ P1: 外部ファイル経由の攻撃面

2.1 外部 KML/GeoJSON 経由の XSS

該当箇所

- js/map.js:341-441 `displayExternalGeoJSON()`
- 外部 KML の `description` を `bindPopup` にそのまま渡す
- `<a href="...">` の `onclick` 書き換えはやっているが、`<script>` タグや `<img onerror="...">`、`<iframe srcdoc="...">` は素通り

影響

- 悪意ある KMZ をユーザーに読み込ませると JS が実行される
- IndexedDB 内の写真・トラックの盗難、Firebase 認証トークンの取得、任意の Firestore 書き込み（§1.2 と組合せ）

現実的シナリオ

- LINE / メールで KMZ ファイルが共有され、ユーザーが Load ボタンで読み込んだ瞬間に攻撃成立

対応

- DOMPurify を導入し `description` の HTML を浄化
- 許可タグ: `<a>`, `<img>`, `<br>`, `<b>`, `<i>`, `<div>`, `<span>` 程度
- 禁止: `<script>`, `<iframe>`, `<object>`, `<embed>`, `on*` 属性, `href="javascript:..."`, `src="javascript:..."`
- 既に行っている `<a onclick>` 書き換えロジックも DOMPurify 通過後に限定

2.2 写真ライトボックスの URL 検証不足

該当箇所

- js/ui-photo.js `buildExternalPhotoList()` で `description` から `https?://` を抽出
- `showDrivePhotoPopup()` で `iframe.src` に流し込む

影響

- 悪意ある KMZ に任意の URL を仕込むことで、フィッシングサイトを iframe 内に表示可能
- ユーザーは「外部写真の表示」と思いつつクレデンシャル等を入力するリスク

対応

- 許可リスト方式（`drive.google.com`, `lh3.googleusercontent.com` 等）
- 不一致なら `window.open(url, '_blank', 'noopener')` でブラウザに委譲

---

3. ◍ P2: 運用・データ保全

3.1 Service Worker のキャッシュ＆更新ハンドリング

問題1: `urlsToCache` の漏れ

- `service-worker.js` の `urlsToCache` に **以下のモジュールが入っていない**
  - `js/auth.js`
  - `js/ui-auth.js`
  - `js/ui-common.js`
  - `js/ui-photo.js`
  - `js/ui-dialog.js`
  - `js/ui-settings.js`
  - `js/kmz-handler.js`
- fetch ハンドラが補完するため通常は動くが、**初回オフライン起動時に欠落する可能性**

問題2: GPS 記録中の自動リロード

- `install` で `skipWaiting()`、`activate` で `clients.claim()` ＋ 旧キャッシュ存在時に `SW_UPDATED` を送信
- クライアント側 `app-main.js` は `SW_UPDATED` を受信すると `window.location.reload()` を実行
- **GPS 記録中にデプロイがあると、ユーザー無確認でページがリロードされる**
- IndexedDB は保持されるが、`previousTotalPoints` などのメモリ状態がリセットされ、Wake Lock も切れる

問題3: `cache.addAll` の脆弱性

- 1 ファイルでも fetch 失敗で install 全体が失敗する
- CDN（unpkg）の一時障害でデプロイが詰まる可能性

対応

- `urlsToCache` に最新の JS モジュール一覧を反映
- `SW_UPDATED` 受信時、`isTracking` 中なら **「次回起動時に新バージョン適用」** とトーストで通知し、即時 reload は抑止
- `cache.addAll` の代わりに個別 `cache.put` ＋ try/catch で耐障害化

3.2 データ消失動線

操作	問題
Clear ボタン	`clearIndexedDBSilent()` は `tracks` / `photos` / `externals` / `external_photos` を **すべて削除**。確認文面が「表示中のルートとマーカー…」と軽く、インポートした外部 GeoJSON も巻き添えで消える
Start → "Start New (Clear Data)"	既存トラック・写真を即削除（外部レイヤーは保持）。**未保存の前回データが失われる**。直前バックアップを促す UI なし
Retake	プレビュー直後に `deletePhoto(currentPhotoId)` を実行してから新規撮影。新撮影に失敗すると元写真も失われる
同名 Save の自動連番	`_2`〜`_100` まで自動採番、超えると一括失敗。100超ケースの救済なし
localStorage 容量	メッセージ履歴・各種設定で軽微だが、IndexedDB と違って quota 例外時のハンドリングなし
IndexedDB 障害時	`initIndexedDB` 失敗で `alert` のみ。プライベートブラウジング iOS Safari ではしばしば失敗

対応

- Clear / Start "Init" 時に「未保存トラック n 件、写真 m 枚があります。先に Save しますか？」を表示
- Retake は **新撮影成功後に旧写真を削除**する順序に変更
- 連番100超は別名提案または失敗扱いを明示

3.3 CDN ライブラリへの SRI 付与漏れ

現状

- `index.html` で読み込む CDN のうち、`integrity` 属性が付いているのは Leaflet CSS/JS のみ
- 未付与: jszip, togeojson, firebase-app-compat, firebase-auth-compat, firebase-firestore-compat, firebase-storage-compat, firebase-functions-compat, Google Fonts

影響

- CDN 改ざん時に任意 JS が実行される
- Service Worker がキャッシュしているため初回のみ顕在化するが、リスクとしては残る

対応

- `integrity="sha384-..."` + `crossorigin="anonymous"` を全 CDN タグに付与
- バージョンアップ時にハッシュも更新

3.4 プライバシー（位置情報）

現状

- 緯度経度を小数点 5 桁（≒1m 精度）で永続保存
- 自宅・職場・通勤経路が事実上特定可能
- §1.2 のルール緩により、現状は他人からも見える状態（P0-2 解消で大幅低減）
- メールアドレスを `userAdmin` に保存し、Cloud Function で送信先に使用
- **退会フロー（`userAdmin` 削除と Firestore データ削除）が未実装**

対応

- 自宅周辺の自動マスク（出発地点を 100m 単位に丸める等）を検討
- Settings に「アカウント削除」を実装し、`userAdmin/{username}` + 自身の `tracks/*` を一括削除
- プライバシーポリシーの明示

---

4. 🟢 P3: 品質・アクセシビリティ

4.1 `meta viewport` のズーム禁止

- `index.html` で `maximum-scale=1.0, user-scalable=no` を指定
- WCAG 的に視覚障害者の操作性を損なう
- 写真ビューア内で独自ピンチズームを実装している影響で全体禁止しているが、地図やテキストのズームも妨げている
- 対応: `user-scalable=yes` に変更し、写真ビューア内のみ独自実装を維持

4.2 `functions/.env` の運用

- `GDRIVE_REFRESH_TOKEN` / `GDRIVE_CLIENT_SECRET` / `SMTP_PASS` が開発者個人 PC にしか存在しない
- 引継ぎ・障害復旧で困難
- 対応: Firebase Functions Configuration（`functions:config:set`）または Google Cloud Secret Manager への移行

4.3 main 直プッシュ運用

- 直近 10 コミットすべて main 直接（PR・レビューなし）
- Service Worker の `CACHE_NAME` 更新漏れが起きた場合、新版がユーザーに伝播しない
- 対応: 軽量な GitHub Actions で「`service-worker.js` の `CACHE_NAME` が更新されたか」を CI チェック

---

5. 着手順序（推奨）

即日（最遅でも当週）

1. **Firebase Console で API キーを再発行**（旧キーを失効）
2. `js/firebase-config.js` を新キーで更新し、`firebase deploy --only hosting`
3. `docs/firestore.rules` / `docs/storage.rules` を `userId` ベースに書き直し、`firebase deploy --only firestore:rules,storage`
4. Cloud Functions の `context.auth` チェックを `userAdmin/{username}.role == 'admin'` 確認に置換し、`firebase deploy --only functions`

数日内

5. `git filter-repo` で `firebase-config.js` を全履歴から削除
6. `.gitignore` を修正し、両リポジトリの Private 化を検討
7. `userAdmin` を含む全 Firestore データを再点検（不正書き込みの痕跡確認）

次のスプリント

8. DOMPurify 導入（外部 KML/GeoJSON サニタイズ）
9. 写真ライトボックス URL の許可リスト化
10. Service Worker の `urlsToCache` 更新＋ GPS 記録中の自動リロード抑止
11. CDN への SRI 付与

継続改善

12. Clear / Start / Retake の確認 UI 強化
13. 退会フロー実装、位置情報マスク機能
14. main ブランチ保護＋ CI チェック

---

6. 参考: P0 三点が同時に解消されない場合の最悪ケース

1. 攻撃者が `RouteLogger-dev` または `RouteLogger` の GitHub リポジトリを発見
2. `js/firebase-config.js` から API キーを取得
3. 任意の HTTP クライアントから Firebase に匿名サインイン
4. **`tracks` コレクションを全件取得**（全ユーザーの GPS 軌跡＋写真メタ＋メールアドレス）
5. Firebase Storage の写真 URL からバイナリを直接ダウンロード（Storage Rules も同条件）
6. 興味があれば `tracks` を全件削除（バックアップなし）
7. `migrateRoutesToDrive({ all: true })` を呼び、Cloud Functions の課金とサーバ側 Drive 容量を消耗させる

これらは **既存ツール（curl + firebase-admin SDK）と公開仕様書のみで実行可能**。攻撃の難易度は低い。

RouteLogger - GPS位置記録 © 2026